

Project Report: Secure Self-Hosted Media Infrastructure in a Restricted Network Environment

AABIRROUCHE Marouane
Network Systems Student

January 4, 2026

1 Introduction and Problem Statement

1.1 The Challenge

Living in a university dormitory presents unique connectivity challenges. While the local network infrastructure exists, internet reliability is often inconsistent. Furthermore, as a resident, I have no administrative access to the network edge equipment (routers/firewalls), making traditional remote access methods (like Port Forwarding) impossible due to NAT (Network Address Translation) restrictions.

1.2 The Objective

To utilize available hardware storage to create a centralized media server capable of:

- Hosting high-quality video and audio files locally.
- Functioning independently of the dormitory's internet stability.
- Providing secure remote access for personal devices outside the local network without exposing ports to the public internet.

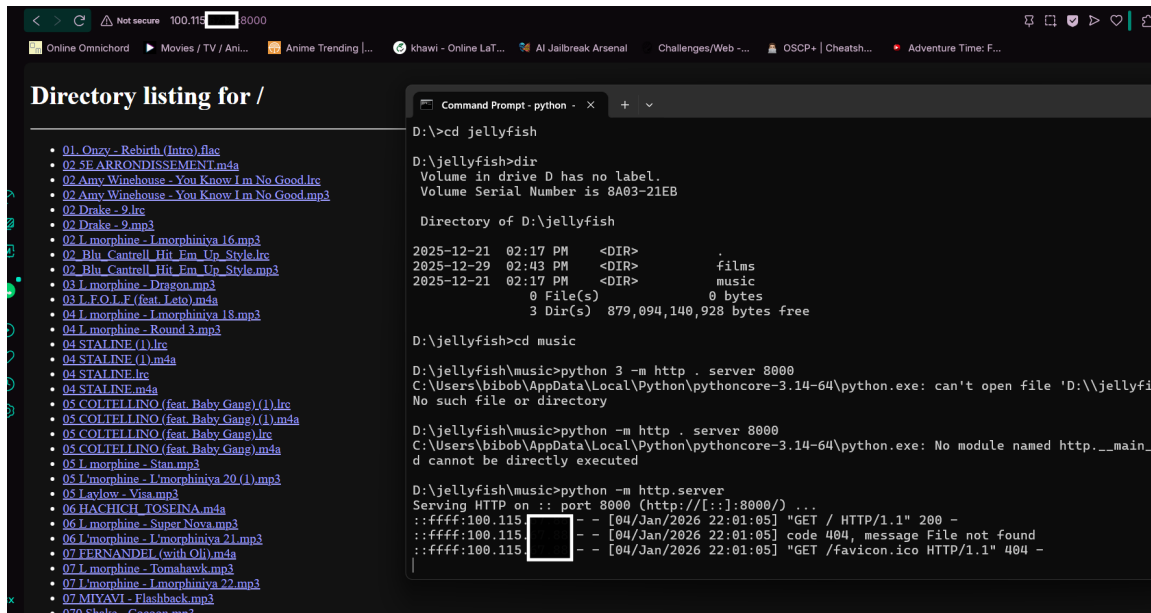
2 Phase 1: Proof of Concept (Python)

Before committing to a complex software stack, I needed to verify the local network's reliability for streaming. I utilized Python's built-in HTTP server for a rapid connectivity test.

Listing 1: Simple HTTP Server Command

```
$ python -m http.server
```

This confirmed that the internal bandwidth was sufficient for streaming, validating the project feasibility.



3 Phase 2: Service Deployment (Jellyfin)

3.1 Selection Process

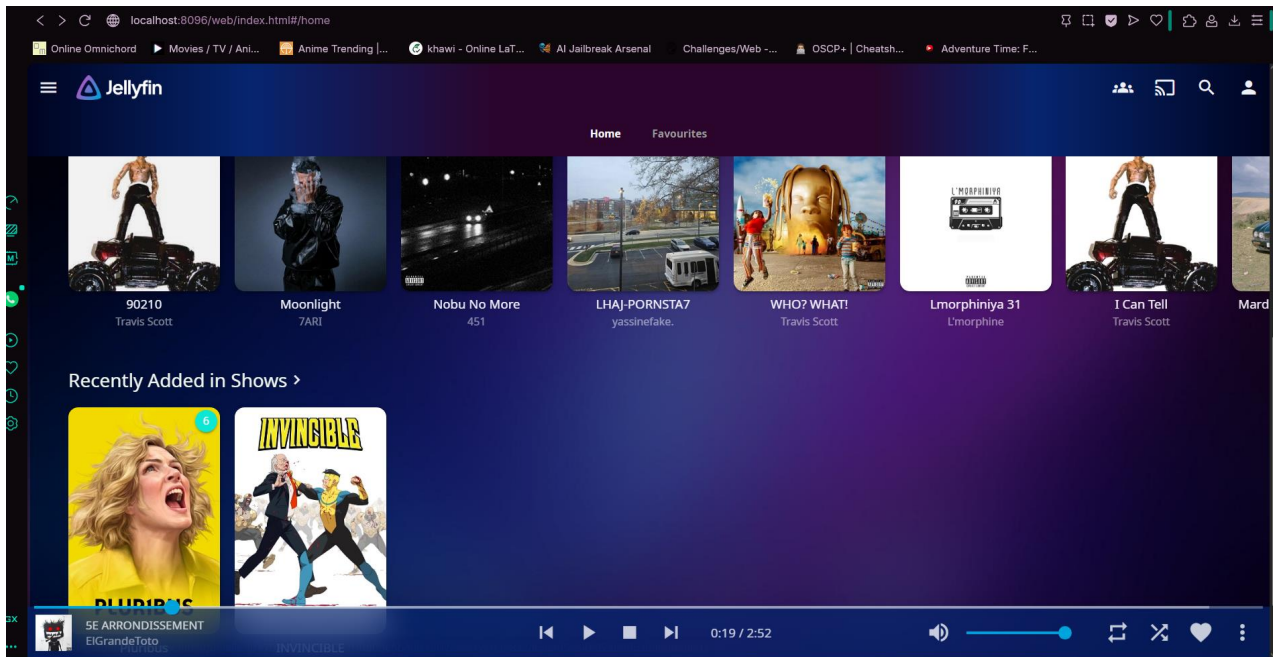
I initially attempted to develop a custom solution using AI-assisted coding tools ("vibe coding"). However, analyzing the time-cost efficiency, I opted for an Open Source solution to prioritize stability and features.

I selected **Jellyfin** because:

- It is Free and Open Source Software (FOSS).
- It handles on-the-fly transcoding.
- It creates a rich metadata interface for the media library.

3.2 Implementation

The server was deployed on a Windows host machine (serving as the main node). The media library was organized, and the Jellyfin service was configured to run in the background, indexing the local storage.



4 Phase 3: Network Traversal Remote Access

4.1 The NAT Problem

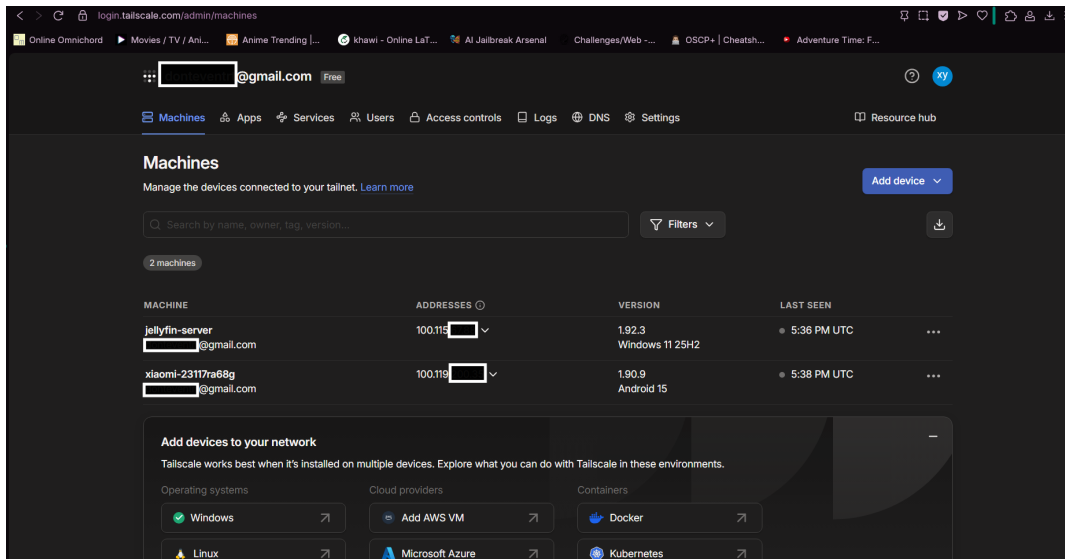
To access the server from my smartphone or laptop while outside the dorm, I faced a "Double NAT" environment. Since I cannot configure Port Forwarding on the dorm router, the server was invisible to the internet.

4.2 The Tailscale Solution

I implemented **Tailscale**, a mesh VPN based on the WireGuard protocol.

How it works in this architecture:

1. Tailscale was installed on the host Server and client devices (Phone/Laptop).
2. It creates a secure, encrypted peer-to-peer overlay network.
3. This assigns a stable private IP address (100.x.y.z) to the server.
4. Traffic is routed securely through the tunnel, bypassing the dorm's firewall restrictions entirely.



5 Future Improvements

- **Migration to Linux:** Moving the server from Windows to a dedicated Linux environment (likely Linux Mint or Ubuntu Server) for better resource management.
- **Automation:** Scripting automated backups for the configuration files.

6 Conclusion

This project successfully solved the connectivity issue by leveraging local storage and modern networking overlays. It demonstrates the practical application of client-server architecture, NAT traversal strategies, and self-hosted service administration.

